**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ,
СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

ОРДЕНА ТРУДОВОГО КРАСНОГО ЗНАМЕНИ
ФЕДЕРАЛЬНОГО ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

**«МОСКОВСКИЙ
ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
СВЯЗИ И ИНФОРМАТИКИ»**

(МТУСИ)

Кафедра «Корпоративные информационные системы»

ЛАБОРАТОРНАЯ РАБОТА № 3

по дисциплине «Прикладные информационные системы»
на тему:

СОЗДАНИЕ ПЕРВОЙ МОДЕЛИ ДАННЫХ И ЕЁ РЕГИСТРАЦИЯ В АДМИНИСТРАТИВНОМ ПРИЛОЖЕНИИИ DJANGO. ДИНАМИЧЕСКОЕ ГЕНЕРИРОВАНИЕ ШАБЛОНА ДЛЯ ВЫВОДА ВСЕХ ЭКЗЕМПЛЯРОВ ЭТОЙ МОДЕЛИ.

Выполнила: студентка группы БСТ2256
Журавлева О.А.

Студенческий билет № 3БСТ22158

Проверил: Сергеев А.Е.

Москва 2026

С О Д Е Р Ж А Н И Е

Цель работы **3**

Задачи **3**

Ход работы **4**

1 Создание модели данных и её регистрация в административном интерфейсе Django **4**

1.1 Создание проекта и приложения 4

1.2 Настройка проекта 4

1.3 Создание омдели Article 4

1.4 Регистрация модели в административном интерфейсе 5

1.5 Создание записей через административную панель и работа с базой данных 6

2 Динамическое генерирование шаблона для вывода всех экземпляров этой модели **8**

2.1 Создание шаблона archive.html 8

2.2 Создание представления archive 8

2.3 Настройка URL 9

2.4 Проверка работы и добавление статей 9

Заключение **12**

Цель работы

Научиться создавать модели данных в Django, регистрировать их в административном интерфейсе, работать с базой данных через админку и напрямую через менеджер SQLite, а также динамически генерировать HTML-страницы на основе данных из модели.

Задачи

1. Создать проект blog и приложение articles.
2. Настроить проект: подключить базу данных SQLite3, добавить приложение в INSTALLED_APPS.
3. Создать модель Article с полями: title, author (внешний ключ к пользователю), text, created_date.
4. Реализовать методы __unicode__ (или __str__) и get_excerpt.
5. Зарегистрировать модель в административном интерфейсе с настройкой отображения.
6. Создать через админку три статьи.
7. С помощью менеджера SQLite (например, SQLite Manager) просмотреть таблицу, изменить текст одной статьи и название другой.
8. Создать шаблон archive.html для вывода всех статей.
9. Создать представление archive, передающее в шаблон список всех статей.
10. Настроить URL для отображения архива на главной странице.

Ход работы

1 Создание модели данных и её регистрация в административном интерфейсе Django

1.1 Создание проекта и приложения

1. В папке lab3 выполним команду создания проекта:

```
django-admin startproject blog
```

2. Перейдем в папку проекта и создадим приложение articles:

```
python manage.py startapp articles
```

1.2 Настройка проекта

1. В файле settings.py укажем базу данных:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': 'db_helloworld',
    }
}
```

2. Добавим приложение articles в INSTALLED_APPS:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'articles'
]
```

3. Выполним миграции для создания таблиц в консоли:

```
python manage.py migrate
```

1.3 Создание омдели Article

1. В файле articles/models.py опишем модель Article.

```
from ast import pattern
class Article(models.Model):
    title = models.CharField(max_length=200)
    author = models.ForeignKey(User, on_delete=models.CASCADE)
    text = models.TextField()
    created_date = models.DateField(auto_now_add=True)
```

```
    def __str__(self):
        return f"{self.author.username}: {self.title}"

    def get_excerpt(self):
        if len(self.text) > 140:
            return self.text[:140] + "..."
        else:
            return self.text
]
```

title – заголовок статьи (строка до 200 символов).

author – внешний ключ к модели User (встроенная модель Django). При удалении пользователя связанные статьи также будут удалены (on_delete=models.CASCADE).

text – текст статьи (неограниченной длины).

created_date – дата создания, заполняется автоматически при создании записи.

__str__ – строковое представление объекта.

get_excerpt – возвращает первые 140 символов текста статьи (для отображения в списке).

2. Выполним создание миграций и применим их:

```
python manage.py makemigrations articles
python manage.py migrate
```

1.4 Регистрация модели в административном интерфейсе

1. Создадим файл articles/admin.py и напишем:

```
from .models import Article

class ArticleAdmin(admin.ModelAdmin):
    list_display = ('title', 'author', 'get_excerpt', 'created_date')

admin.site.register(Article, ArticleAdmin)
```

ArticleAdmin описывает, как модель отображается в админке: в списке статей показываются поля title, author, результат метода get_excerpt и created_date.

2. Создадим суперпользователя для входа в административную панель:

```
python manage.py createsuperuser
```

1.5 Создание записей через административную панель и работа с базой данных

1. Запустим сервер разработки:

```
python manage.py runserver
```

2. Перейдем по адресу http://127.0.0.1:8000/ и войдем под суперпользователем (рис. 1).

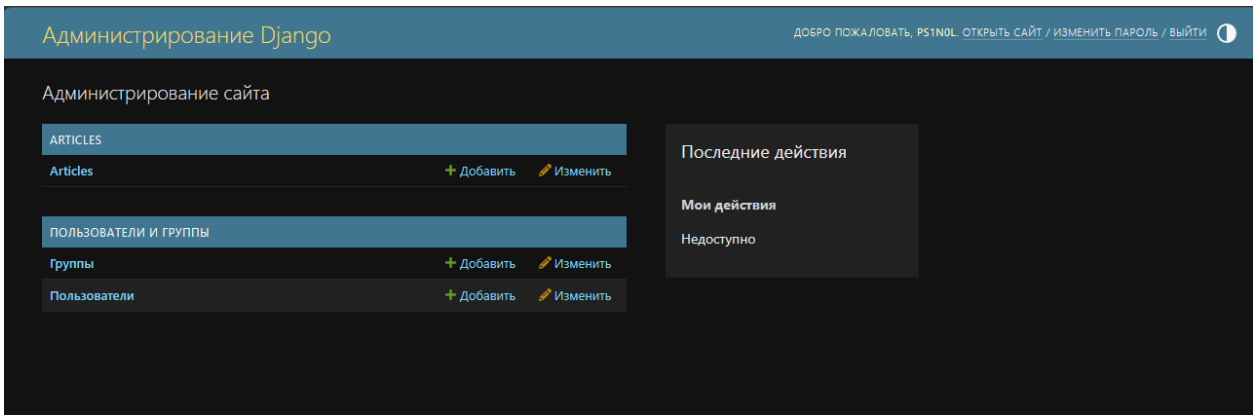


Рисунок 1 – страница http://127.0.0.1:8000/admin/.

3. В разделе «Articles» нажмём кнопку «Добавить» и создадим три статьи, заполнив поля (рис. 3).

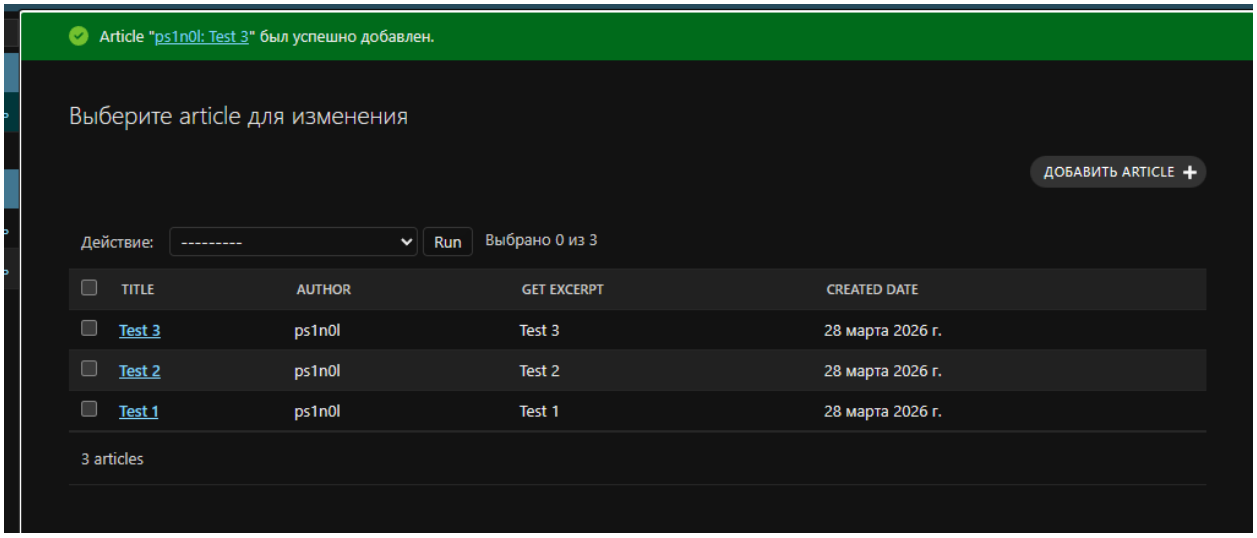


Рисунок 3 – добавленные статьи.

3. С помощью программы управления базами данных SQLiteManager откроем файл базы данных текущего проекта. Найдем созданные

в предыдущем пункте задания экземпляры записей. Изменим текст одной записи и название статьи для другой (рис. 4).

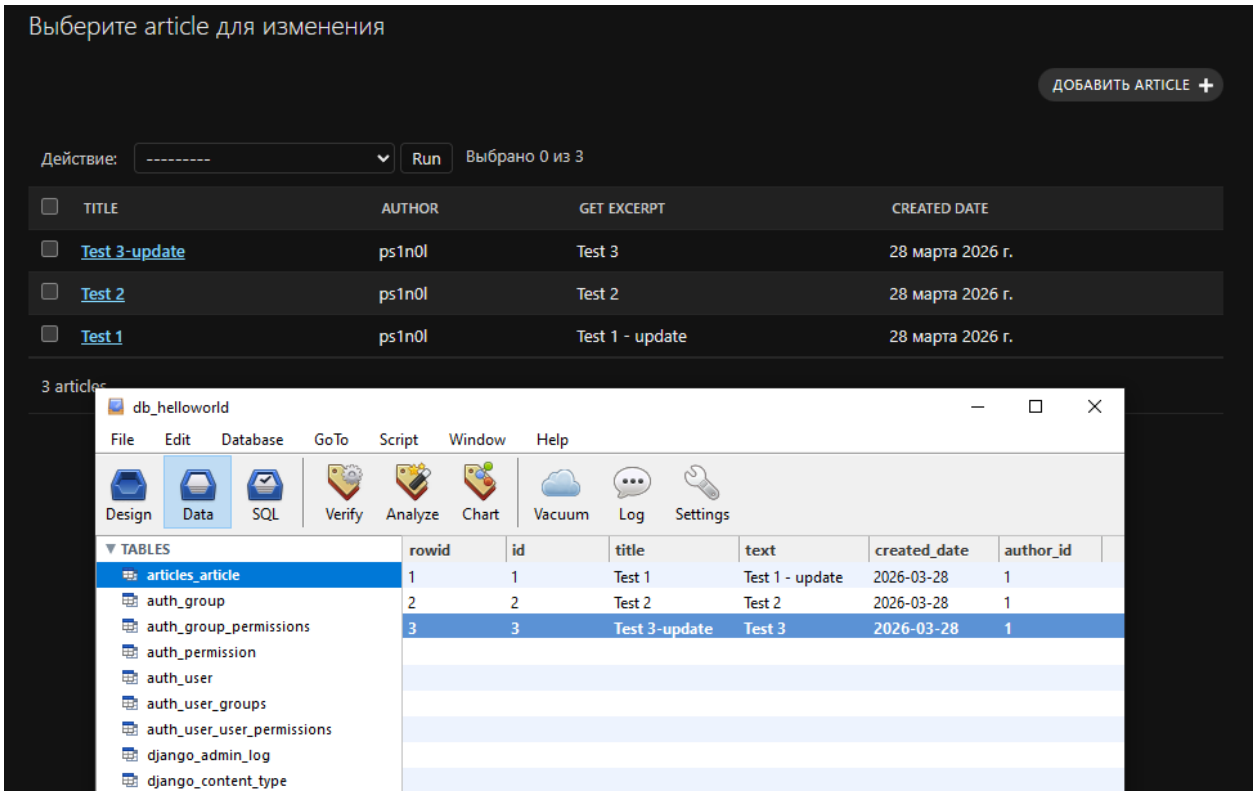


Рисунок 4 – измененные статьи через SQLite Manager.

2 Динамическое генерирование шаблона для вывода всех экземпляров этой модели

2.1 Создание шаблона archive.html.

1. Создадим папку templates внутри приложения articles, внутри которого создадим файл archive.html.

2. Внутрь acrhive.html добавим код:

```
<!DOCTYPE html>
<!DOCTYPE html>
<html>
    <head>
        <title>Архив всех статей</title>
    </head>
    <body>
        <div class="header">
            <img src="{{ STATIC_URL }}img/dpb-logo.png" />
        </div>
        <div class="archive">
            {% for post in posts %}
            <div class="one-post">
                <h2 class="post-title">{{ post.title }}</h2>
                <div class="article-info">
                    <div class="article-author">{{ post.author.username }}</div>
                    <div class="article-created-date">{{ post.created_date }}</div>
                </div>
                <p class="article-text">{{ post.get_excerpt }}</p>
            </div>
            {% endfor %}
        </div>
    </body>
</html>
```

Цикл {% for post in posts %} перебирает все переданные статьи и отображает каждую.

2.2 Создание представления archive

1. В файле articles/views.py напишем:

```
from django.shortcuts import render
from .models import Article

def archive(request):
    posts = Article.objects.all()  # получаем все статьи из БД
    return render(request, 'archive.html', {'posts': posts})
```

Article.objects.all() – запрос к базе данных, возвращающий все записи модели Article.

2.3 Настройка URL

В файле urls.py подключим представление archive для корневого URL:

```
from django.contrib import admin
from django.urls import path
from articles import views

urlpatterns = [
    path('', views.archive, name='archive'),
    path('admin/', admin.site.urls),
]
```

2.4 Проверка работы и добавление статей

1. Запустим сервер и перейдем на http://127.0.0.1:8000/ (рис. 5).

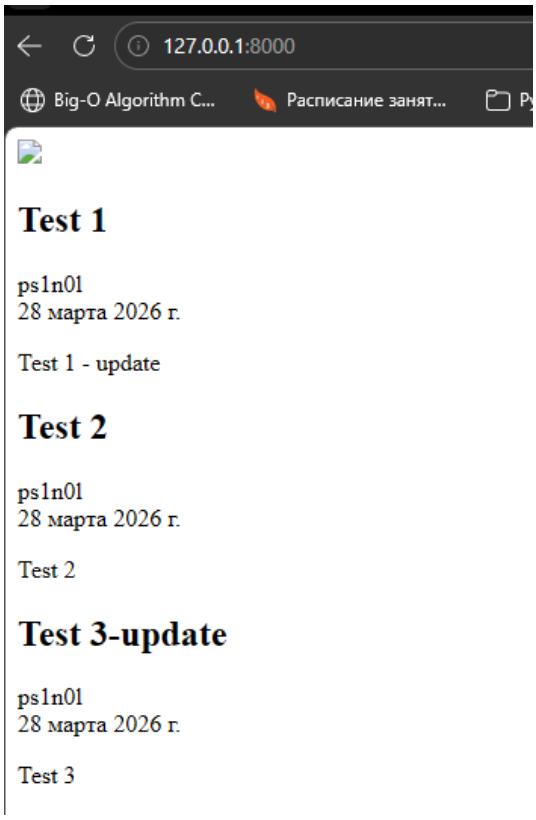


Рисунок 5 – страница http://127.0.0.1:8000/.

2. Вновь откроем файл базы данных sqlite3, где хранятся экземпляры статей к текущему проекту. Добавим новую запись в блог непосредственно через менеджер базы данных (рис. 6-8).

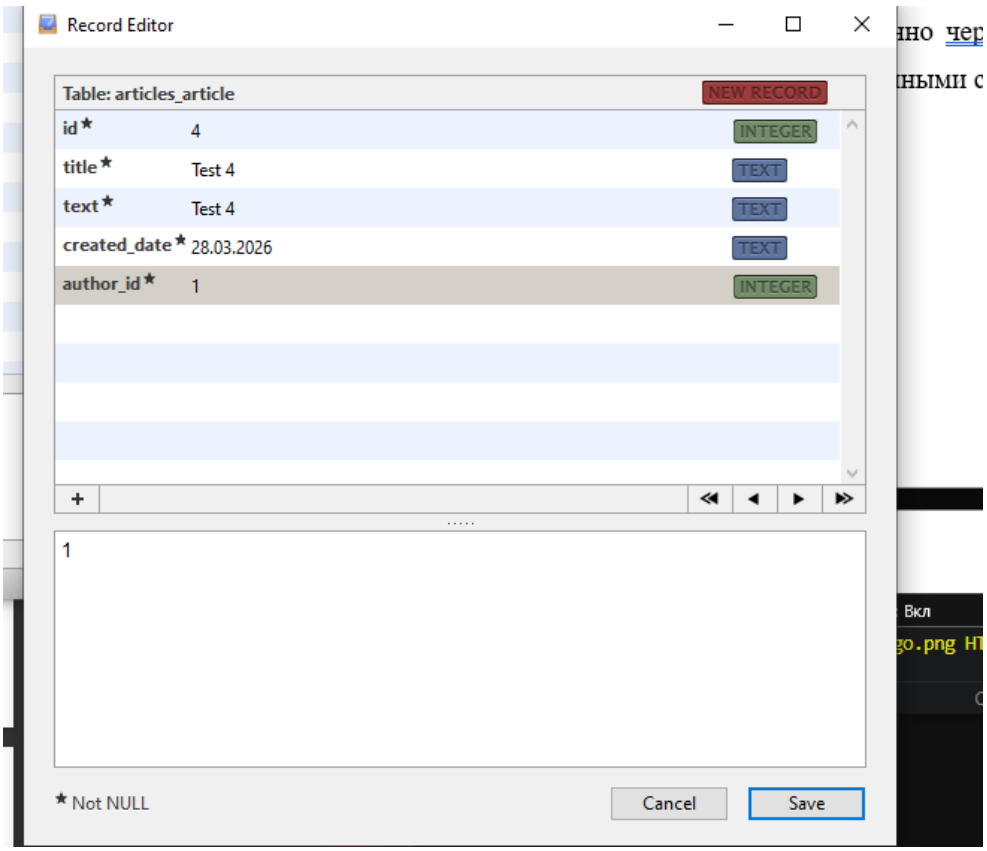
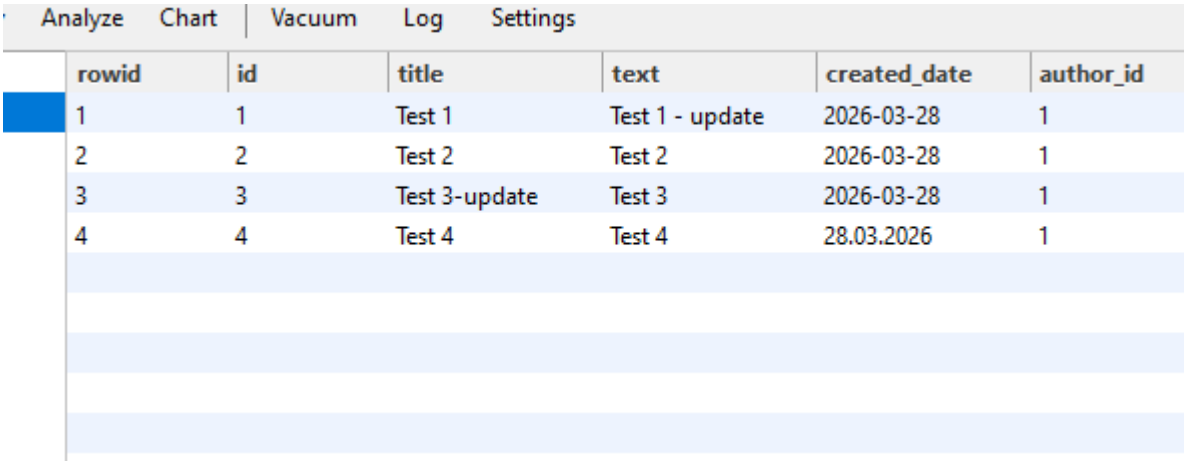


Рисунок 6 – добавление новой записи.



Analyze Chart Vacuum Log Settings

rowid	id	title	text	created_date	author_id
1	1	Test 1	Test 1 - update	2026-03-28	1
2	2	Test 2	Test 2	2026-03-28	1
3	3	Test 3-update	Test 3	2026-03-28	1
4	4	Test 4	Test 4	28.03.2026	1

Рисунок 7 – список существующих записей.

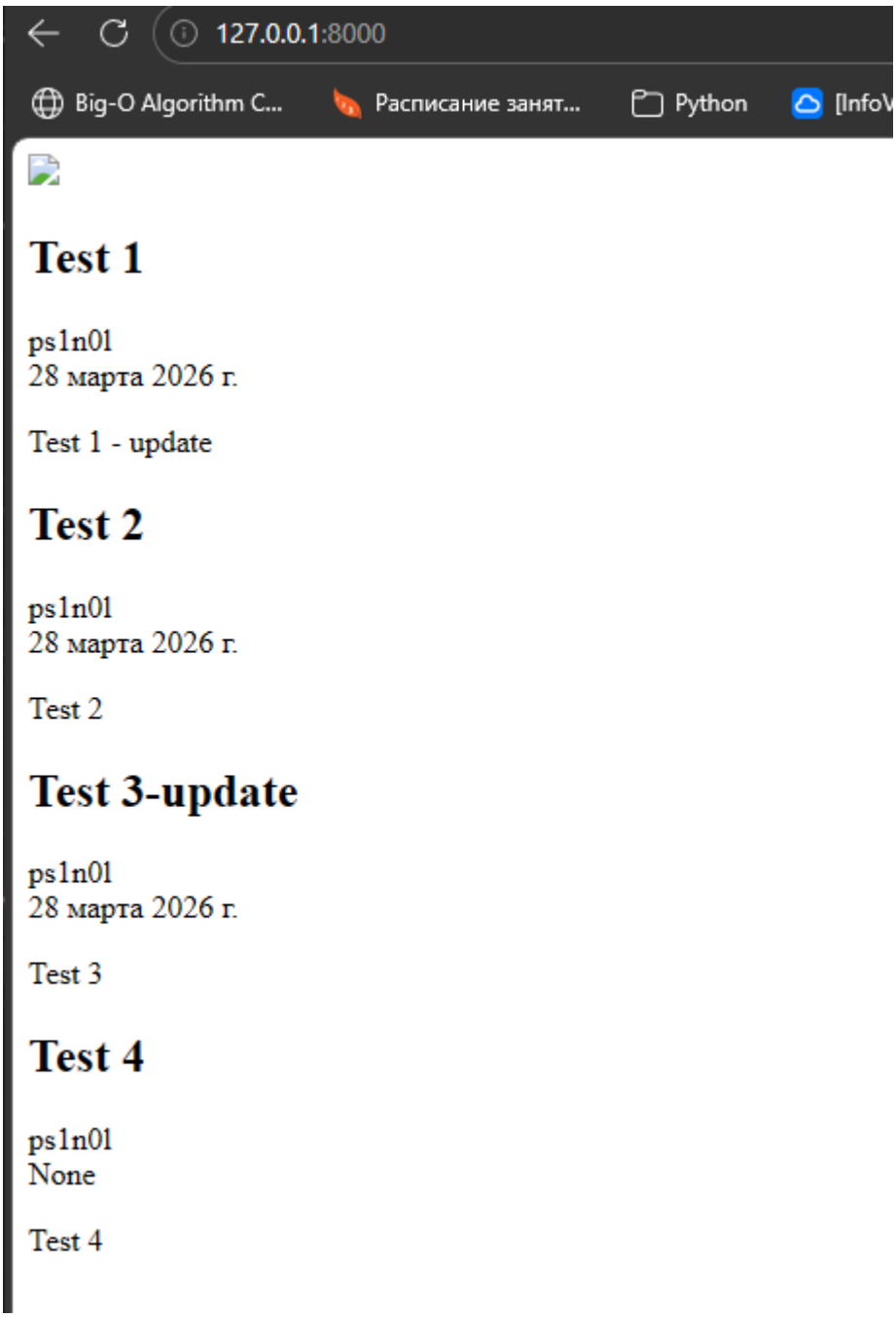


Рисунок 8 – обновленная страница http://127.0.0.1:8000/.

Заключение

В ходе лабораторной работы были выполнены все поставленные задачи: Создан проект blog и приложение articles, выполнена настройка проекта: подключена SQLite3, приложение добавлено в INSTALLED_APPS, создана модель Article с полями: title, author, text, created_date, реализованы методы __str__ и get_excerpt для удобного отображения. Модель зарегистрирована в административном интерфейсе с кастомизацией списка записей. Через административный интерфейс созданы три статьи. С помощью менеджера SQLite выполнены изменения существующих записей (текст и название), а также добавлена новая запись напрямую через менеджер базы данных. Создан шаблон archive.html, использующий цикл для вывода всех статей, с подключением статического логотипа. Разработано представление archive, передающее в шаблон список всех статей, и настроен URL для отображения на главной странице.